

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing techniques. (BL4,BL5)
Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:40

Annexure A

MOCK INTERVIEW

Code of Conduct:

*I **D Varshini / 192320007** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Mock Interview – Sample Questions (5 Questions)

Mock Interview Test Format for Object Oriented Analysis and Design (OOAD)

Section A	Self Introduction & Fundamentals	3 Minutes	5
Section B	Technical Questions (OOAD Concepts)	7 Minutes	10
Section C	UML Modeling & Design	7 Minutes	10
Section D	Case Study / Problem Solving	8 Minutes	10
Section E	HR & Career-Oriented Questions	5 Minutes	5

SECTION A – SELF INTRODUCTION (5 Marks)

Evaluation Criteria

- Communication Skills
- Confidence
- Technical Awareness
- Academic Background

Sample Questions

1. Tell me about yourself.

Answer: I am an engineering student with a strong interest in software development, object-oriented design, and AI-based applications. I enjoy converting requirements into structured solutions using classes, UML models, and modular code. I have worked with programming languages such as Python, Java, C, and SQL, and I am continuously improving my problem-solving and software engineering skills through projects and practical exercises.

2. Why did you choose Computer Science?

Answer: I chose Computer Science because it combines logical problem solving with creativity. It allows me to design solutions for real-world problems, automate repetitive tasks, and build systems that can scale from a simple application to a large platform. I also like that the field keeps evolving, which gives me continuous opportunities to learn new technologies.

3. Explain your favorite subject in OOAD.

Answer: My favorite part of OOAD is UML modeling, especially class and sequence diagrams. UML helps me visualize a system before implementation by showing its objects, responsibilities, relationships, and interactions. This reduces ambiguity in requirements and makes the final design easier to communicate, review, and maintain.

4. What software development tools have you used?

Answer: I have used tools such as Visual Studio Code for development, Git and GitHub for version control, and database tools for working with SQL. I also use debugging, testing, and command-line tools during development. For object-oriented projects, I use UML diagrams to plan classes, interactions, and system structure before coding.

5. Describe a project where you applied object-oriented concepts.

Answer: In a Smart Farm Monitoring System project, I represented entities such as sensors, fields, readings, and alerts as separate objects with clear responsibilities. Encapsulation protected sensor state, inheritance could be used for different sensor types, and polymorphism allowed different sensors to process readings through a common interface. This structure made the project easier to extend when new sensor types or monitoring rules were added.

SECTION B – TECHNICAL QUESTIONS (15 Marks)

Basic Level

Q1. What is Object-Oriented Analysis and Design?

Answer: Object-Oriented Analysis and Design (OOAD) is a software development approach that models a system as interacting objects. Object-Oriented Analysis focuses on understanding requirements and identifying domain objects, while Object-Oriented Design transforms those findings into classes, interfaces, relationships, methods, and an implementable architecture. OOAD improves modularity, reuse, maintainability, and traceability from requirements to code.

Q2. Differentiate between Analysis and Design.

Answer: Analysis describes what the system must do, whereas design describes how the system will do it. During analysis, developers identify requirements, actors, use cases, domain objects, and business rules without concentrating on implementation details. During design, they decide classes, interfaces, data structures, design patterns, database interaction, and component communication. In short, analysis is problem-oriented and design is solution-oriented.

Q3. Explain Encapsulation with an example.

Answer: Encapsulation is the practice of combining data and the operations that work on that data inside a class while restricting direct access to the internal state. For example, a BankAccount class can keep balance as a private attribute and provide deposit() and withdraw() methods. External code cannot modify balance arbitrarily; it must use methods that can validate the transaction, thereby protecting object integrity.

Q4. What is Inheritance? Mention its types.

Answer: Inheritance allows a new class to acquire and extend the attributes and behavior of an existing class. The existing class is the superclass or parent, and the derived class is the subclass or child. Common types are single inheritance, multilevel inheritance, hierarchical inheritance, multiple inheritance, and hybrid inheritance. Languages such as Java support multiple inheritance of type through interfaces rather than multiple inheritance of classes.

Q5. Define Polymorphism and its benefits.

Answer: Polymorphism means one interface can represent many forms of behavior. For example, a Payment reference can invoke pay(), while CreditCardPayment, UPIPayment, and NetBankingPayment implement pay() differently. Its benefits include flexible code, easier extension, reduced conditional logic, better reuse, and the ability to replace implementations without changing client code.

Q6. What is Abstraction?

Answer: Abstraction exposes only the essential features of an object while hiding unnecessary implementation details. It allows users of a class to work with a clear interface without knowing how the operation is internally performed. For example, a user can call `printer.printDocument()` without knowing the low-level driver commands used to communicate with the printer.

Q7. Difference between Aggregation and Composition.

Answer: Aggregation and composition are both whole-part relationships, but they differ in ownership and lifetime. Aggregation is a weak 'has-a' relationship in which the part can exist independently of the whole, such as a Department having Professors. Composition is a strong ownership relationship in which the part's lifetime normally depends on the whole, such as an Order containing OrderItems. Deleting a department does not necessarily delete professors, while deleting an order usually removes its order items.

Q8. What is Coupling and Cohesion?

Answer: Coupling measures the degree of dependency between modules or classes, while cohesion measures how closely related the responsibilities within a single module are. Good object-oriented design aims for low coupling and high cohesion. This makes classes easier to understand, test, reuse, replace, and modify because each class has a focused purpose and minimal knowledge of other classes.

Q9. What is a Class and an Object?

Answer: A class is a blueprint that defines the attributes and behaviors of a type, whereas an object is a runtime instance of that class. For example, Car may be a class with attributes such as `registrationNumber` and `speed` and methods such as `start()` and `brake()`. A specific car with a particular registration number is an object created from the Car class.

Q10. Explain Object Responsibility.

Answer: Object responsibility describes what an object knows and what it is expected to do within a system. Responsibilities include maintaining its own state, performing operations related to that state, and collaborating with other objects when required. Assigning focused responsibilities according to principles such as high cohesion prevents large 'god classes' and creates a cleaner, more maintainable design.

SECTION C – UML MODELING (15 Marks)

Q11.What is UML?

Answer: Unified Modeling Language (UML) is a standard visual language used to specify, visualize, design, and document software systems. It provides structural diagrams such as class, component, and deployment diagrams, and behavioral diagrams such as use case, sequence, activity, and state diagrams. UML gives developers and stakeholders a common way to communicate system structure and behavior before and during implementation.

Q12.When would you use a Use Case Diagram?

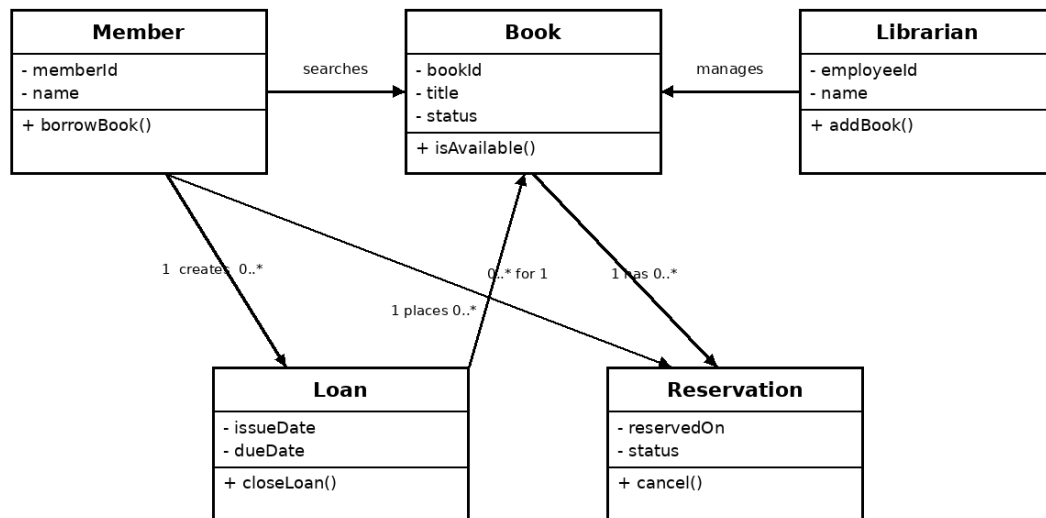
Answer: A Use Case Diagram is used during requirements analysis to show how external actors interact with a system and what major services the system provides. It is especially useful when defining system scope, identifying functional requirements, discussing requirements with non-technical stakeholders, and discovering relationships such as include, extend, and actor generalization.

Q13.Draw and explain a Class Diagram for a Library Management System.

Answer: A Library Management System can be modeled using classes such as Member, Book, Librarian, Loan, and Reservation. Member searches for and borrows books, Librarian manages book records, Loan represents an issued book with issue and due dates, and Reservation records a member's request for an unavailable book. The associations and multiplicities make the

responsibilities and relationships explicit, which helps guide implementation and database design.

Library Management System - Class Diagram



Q14.Differentiate Sequence Diagram and Activity Diagram.

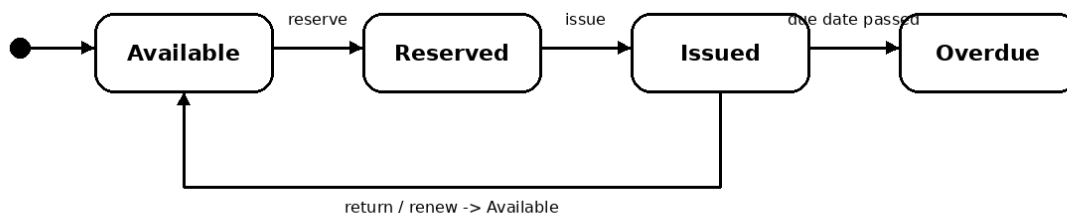
Answer: A Sequence Diagram emphasizes interactions between objects in time order. It shows lifelines, messages, activations, and the sequence in which operations occur, so it is useful for understanding a specific use-case scenario. An Activity Diagram emphasizes workflow and control flow; it shows activities, decisions, parallel paths, and start/end points, making it useful for modeling business processes or algorithms. Thus, sequence diagrams answer 'who communicates with whom and when,' while activity diagrams answer 'what workflow is followed.'

Q15.Explain State Diagram with an example.

Answer: A State Diagram models the different states of an object and the events that cause transitions between those states. For a library book, states may include Available, Reserved,

Issued, and Overdue. A reserve event changes Available to Reserved, issuing changes it to Issued, and passing the due date can change Issued to Overdue. Returning or renewing the book causes another state transition. State diagrams are useful when an object's behavior depends strongly on its current state.

Book Loan - State Diagram



Q16.What is a Deployment Diagram?

Answer: A Deployment Diagram is a UML structural diagram that shows the physical arrangement of software artifacts on hardware or execution nodes. It can represent client devices, application servers, database servers, cloud services, and the communication links between them. It is useful for planning distributed systems, deployment topology, scalability, network communication, and infrastructure responsibilities.

Q17.How do Component Diagrams help software architects?

Answer: Component Diagrams show the major software components, their provided and required interfaces, and dependencies. They help architects divide a large system into manageable modules, understand integration points, identify dependency problems, and plan independent development or deployment. They are also useful for communicating high-level architecture without exposing every class-level detail.

Q18.What is Multiplicity in UML?

Answer: Multiplicity specifies how many instances of one class may be associated with an instance of another class. Common values are 1 for exactly one, 0..1 for optional one, * or 0..* for any number, and 1..* for one or more. For example, one Customer may place 0..* Orders, while each Order belongs to exactly 1 Customer.

SECTION D – CASE STUDY / PROBLEM SOLVING (10 Marks)

Case Study 1

A hospital wants a system to manage:

- Patients
- Doctors
- Appointments
- Billing

Questions

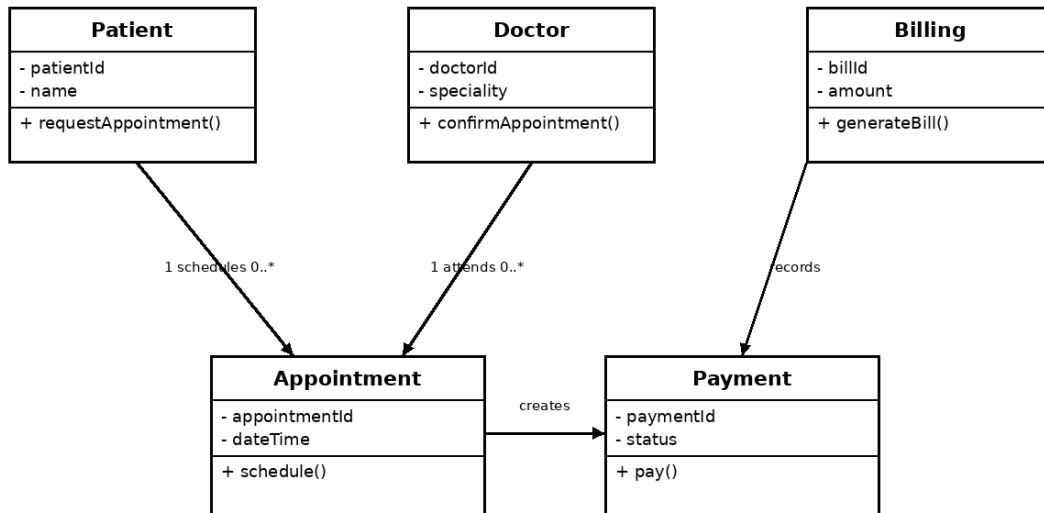
1. Identify the classes.

Answer: The main classes are Patient, Doctor, Appointment, and Billing. Supporting classes may include Payment, MedicalRecord, Department, and Receptionist depending on the required scope. Each class should contain only the data and operations that belong to its responsibility.

2. Draw a simple class diagram.

Answer: A simple design connects Patient and Doctor through Appointment, because an appointment represents the scheduled interaction between them. Billing is associated with the patient or appointment, and Payment may be used to record settlement of a bill. The following class diagram shows this structure.

Hospital System - Simple Class Diagram



3. Suggest relationships among classes.

Answer: A Patient can have zero or many Appointments, and a Doctor can attend zero or many Appointments; each Appointment refers to one Patient and one Doctor. An Appointment can generate a Bill, and a Bill can be settled through a Payment. Composition may be used when a record has no meaningful lifetime outside its owner, while ordinary associations are suitable for independent domain entities.

4. Which UML diagrams would you use for analysis?

Answer: I would use a Use Case Diagram to capture actors and hospital services, a Class Diagram to model domain entities and relationships, a Sequence Diagram to analyze interactions for scenarios such as booking an appointment, and an Activity Diagram to model workflows such as registration or billing. A State Diagram can also be useful for objects whose status changes, such as Appointment or Bill.

5. Explain how inheritance can be applied.

Answer: Inheritance can be applied by creating a general Person class containing common details such as `personId`, `name`, `phone`, and `address`. Patient, Doctor, and Staff can inherit from Person and add role-specific attributes and behavior. This avoids duplication while preserving

specialized responsibilities, provided the subclasses truly satisfy an 'is-a' relationship with Person.

Case Study 2

An Online Shopping System supports:

- Customers
- Products
- Cart
- Orders
- Payments

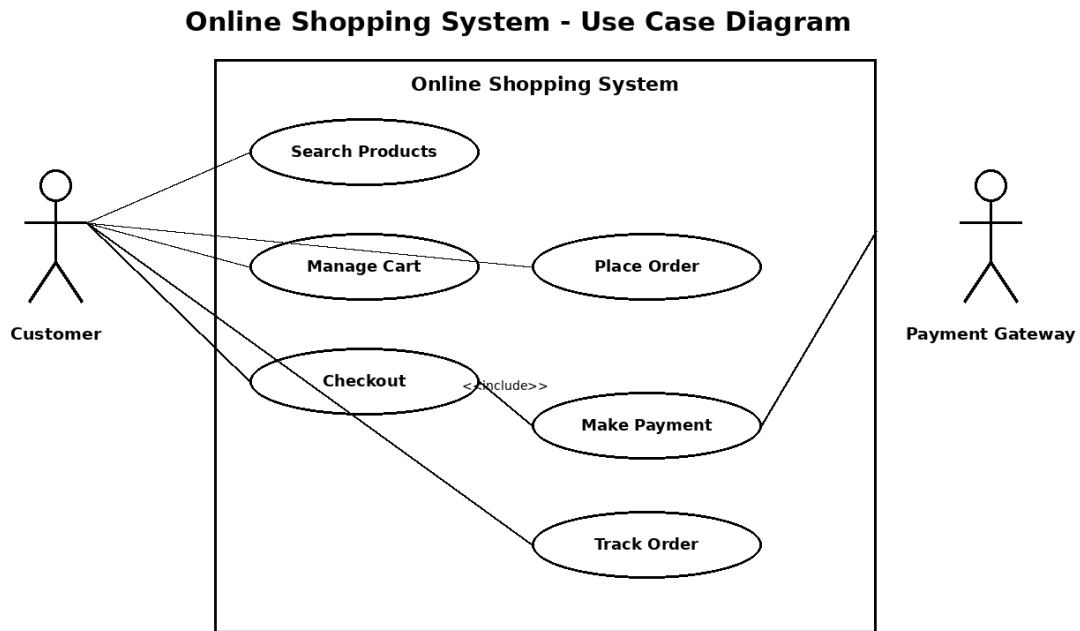
Questions

1. Identify actors and use cases.

Answer: The primary actor is the Customer. External actors can include the Payment Gateway and, depending on scope, an Administrator or Delivery Service. Main use cases include Register/Login, Search Products, View Product, Manage Cart, Checkout, Place Order, Make Payment, Track Order, and Cancel/Return Order. The administrator can manage products, inventory, and order status.

2. Design a use case diagram.

Answer: The use case diagram places the shopping platform inside the system boundary and connects the Customer to product search, cart management, checkout, order placement, and order tracking. The Payment Gateway participates in Make Payment. Checkout can include payment processing and order creation, making the dependencies between user-visible functions clear.



3. Suggest suitable design patterns.

Answer: Suitable patterns include Strategy for interchangeable payment methods, Factory Method for creating payment handlers or shipping services, Observer for order-status notifications, Repository/DAO for data access, and MVC for separating interface, application behavior, and data. These patterns reduce coupling and make it easier to add new payment options, notification channels, or storage technologies.

4. Explain how polymorphism can be used in payment processing.

Answer: A common PaymentMethod interface can define an operation such as pay(amount). Classes such as CardPayment, UPIPayment, WalletPayment, and NetBankingPayment implement the same operation differently. Checkout works with the PaymentMethod abstraction rather than checking each payment type with condition statements. A new method can therefore be added by creating another implementation without changing the checkout logic.

SECTION E – HR & PROFESSIONAL QUESTIONS (5 Marks)

Q19. Why should we hire you?

Answer: You should hire me because I bring a strong learning mindset, structured problem-solving ability, and a solid foundation in programming and object-oriented design. I try to understand requirements clearly, break problems into manageable parts, and communicate my approach rather than only producing code. As a fresher, I am also adaptable and willing to learn the tools, standards, and domain knowledge required by the team.

Q20.What are your strengths and weaknesses?

Answer: My strengths are analytical thinking, curiosity, persistence in debugging, and the ability to learn new technical concepts quickly. I also like organizing a problem before implementing it, which helps reduce avoidable errors. One weakness is that I can spend too much time refining a solution when a simpler version already meets the requirement; I am improving this by time-boxing tasks, prioritizing acceptance criteria, and separating essential improvements from optional ones.

Q21.How do you handle project deadlines?

Answer: I first break the work into smaller tasks, identify dependencies, and prioritize items that affect the critical path. I set intermediate milestones, track progress regularly, and test incrementally rather than leaving integration until the end. If a risk may affect the deadline, I communicate it early and propose a practical alternative or reduced scope instead of waiting until the final day.

Q22.Have you worked in a team project? Explain your role.

Answer: Yes. In team projects, I usually contribute to technical implementation, debugging, integration, and documenting how different modules work together. I make sure my part has a clear interface so teammates can integrate it easily, and I use version control to keep changes traceable. I also discuss blockers early and help test the complete workflow rather than focusing only on my individual module.

Q23.Where do you see yourself in five years?

Answer: In five years, I want to be a dependable software engineer who can take ownership of complete features, make sound design decisions, and contribute to production-quality systems. I would like to deepen my expertise in software engineering and AI-enabled applications while gradually taking more responsibility for architecture, mentoring, and technical decision-making.

RUBRICS FOR CALCULATION:
MOCK INTERVIEW

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge

Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately